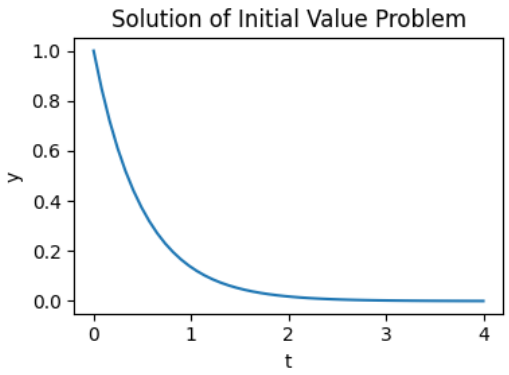


Let us integrate a more complex ODE: a damped spring-mass oscillator. The position of a mass attached to a spring obeys the 2nd order ODE $\ddot{y} + 2\zeta\omega_0\dot{y} + \omega_0^2 y = 0$ with natural frequency $\omega_0 = \sqrt{k/m}$, damping ratio $\zeta = c/(2m\omega_0)$, spring constant k, mass m, and damping coefficient c.

Before using `scipy.integrate.solve_ivp()`, the 2nd order ODE needs to be transformed into a system of first-order ODEs. Note that

$$\frac{dy}{dt} = \dot{y}\frac{d\dot{y}}{dt} = \ddot{y} = -(2\zeta\omega_0\dot{y} + \omega_0^2 y)$$

If we define $z = [z_0, z_1]$ where $z_0 = y$ and $z_1 = \dot{y}$, then the first order equation:

$$\frac{dz}{dt} = \begin{bmatrix} \frac{dz_0}{dt} \\ \frac{dz_1}{dt} \end{bmatrix} = \begin{bmatrix} z_1 \\ -(2\zeta\omega_0 z_1 + \omega_0^2 z_0) \end{bmatrix}$$

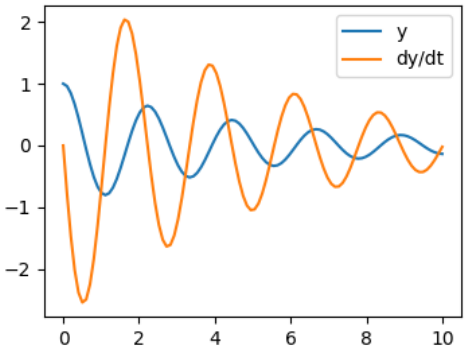
is equivalent to the original second order equation.

We set:

```
>>> m = 0.5  # kg
>>> k = 4  # N/m
>>> c = 0.4  # N s/m
>>> zeta = c / (2 * m * np.sqrt(k/m))
>>> omega = np.sqrt(k / m)
```

and define the function that computes $\dot{z} = f(t, z(t))$:

```
>>> def f(t, z, zeta, omega):
...     return (z[1], -2.0 * zeta * omega * z[1] - omega**2 * z[0])
```



Integration of the system follows:

```
>>> t_span = (0, 10)
>>> t_eval = np.linspace(*t_span, 100)
```

(continues on next page)